

词向量构建与评价：基于SVD与SGNS的方法比较研究

王殿云 2022211733号 2022219107班

1. 实验要求

本实验旨在分别基于SVD（Singular Value Decomposition）分解以及SGNS（Skip-Gram Negative Sampling）两种方法构建词向量并评测其性能。具体要求如下：

1. 使用lmtraining.txt作为词向量学习语料
2. 使用wordsim353_agreed.txt作为评测数据集，通过计算词向量间的余弦相似度与人工标注的相似度进行比较
3. 基于SVD分解的方法：使用上下文窗口K=5，先基于共现矩阵获取高维表示，再采用SVD降维获得词向量
4. 基于SGNS的方法：使用上下文窗口K=2，采用skip-gram模型构建词向量
5. 对两种方法获得的词向量进行评估并比较性能

2. 实验原理

2.1 基于SVD的词向量

分布式语义假设认为"相似的词出现在相似的上下文中"。基于SVD的词向量方法首先统计词与上下文的共现信息，构建共现矩阵，再通过矩阵分解降维获得低维连续向量表示。

2.1.1 共现矩阵

给定词表大小为 V ，共现矩阵 $M \in \mathbb{R}^{V \times V}$ 中的每个元素 M_{ij} 表示词 w_i 与词 w_j 在语料中的共现频次。具体地，在本实验中，我们考虑一个固定大小为 K 的窗口，对于语料中的每个词 w_i ，统计其窗口内出现的上下文词 w_j ，并增加 M_{ij} 计数，同时添加距离加权，使得越远的上下文权重越低：

$$M_{ij} = \sum_{w_i, w_j \text{ co-occur}} \frac{1}{|pos(w_i) - pos(w_j)|}$$

其中 $pos(w)$ 表示词 w 在语料中的位置。

2.1.2 PPMI变换

共现矩阵通常存在很强的频率偏差，常见词的共现频次往往远高于罕见词。为了解决这个问题，通常使用Positive Pointwise Mutual Information (PPMI)变换：

$$\text{PPMI}(w_i, w_j) = \max \left(0, \log \frac{P(w_i, w_j)}{P(w_i)P(w_j)} \right)$$

其中：

- $P(w_i, w_j)$ 是词 w_i 和 w_j 共现的概率，可由 $M_{ij} / \sum_{i,j} M_{ij}$ 估计
- $P(w_i)$ 是词 w_i 的边缘概率，可由 $\sum_j M_{ij} / \sum_{i,j} M_{ij}$ 估计
- $P(w_j)$ 是词 w_j 的边缘概率，可由 $\sum_i M_{ij} / \sum_{i,j} M_{ij}$ 估计

PPMI变换强调了具有信息量的共现关系，减弱了频率偏差的影响。

2.1.3 奇异值分解(SVD)

应用PPMI变换后，得到矩阵 M^{PPMI} ，然后通过奇异值分解降维：

$$M^{PPMI} = U \Sigma V^T$$

其中 U 和 V 是正交矩阵， Σ 是对角矩阵，对角线上的元素称为奇异值，按从大到小排列。取前 d 个最大的奇异值及其对应的奇异向量，得到降维后的词向量：

$$W = U_d \Sigma_d^{1/2}$$

其中 U_d 是 U 的前 d 列， Σ_d 是 Σ 的左上角 $d \times d$ 子矩阵。

2.2 基于SGNS的词向量

Skip-Gram Negative Sampling (SGNS)是基于神经网络的词向量学习方法，由Word2Vec提出。

2.2.1 Skip-Gram模型

Skip-Gram模型的目标是预测给定中心词周围的上下文词。对于中心词 w_t ，模型最大化以下对数似然：

$$\sum_{-c \leq j \leq c, j \neq 0} \log P(w_{t+j} | w_t)$$

其中 c 是窗口大小，条件概率 $P(w_{t+j} | w_t)$ 通过softmax函数定义：

$$P(w_o | w_i) = \frac{\exp(v_{w_o}^T \cdot v_{w_i})}{\sum_{w \in V} \exp(v_w^T \cdot v_{w_i})}$$

其中 v_{w_i} 是词 w_i 的目标词向量， v_{w_o} 是词 w_o 的上下文词向量。

2.2.2 负采样

在大规模词表上，计算softmax的归一化项计算成本很高。负采样通过近似这个优化问题：对于每个正样本(中心词-上下文词对)，采样 k 个负样本(不在上下文中的词)，最大化以下目标：

$$\log \sigma(v_{w_o}^T \cdot v_{w_i}) + \sum_{j=1}^k \mathbb{E}_{w_j \sim P_n(w)} [\log \sigma(-v_{w_j}^T \cdot v_{w_i})]$$

其中 σ 是sigmoid函数， $P_n(w)$ 是负样本的采样分布，通常设为词频的3/4次方。

2.2.3 下采样

为了减少高频词的过度影响，SGNS采用下采样策略，以概率 $P(w_i) = 1 - \sqrt{\frac{t}{f(w_i)}}$ 丢弃词 w_i ，其中 t 是阈值参数， $f(w_i)$ 是词 w_i 的频率。

3. 实验环境

本实验使用AutoDL平台提供的算力租赁服务，具体使用的服务器配置如下：

- PyTorch 2.0.0
- Python 3.8(ubuntu20.04)
- CUDA 11.8
- RTX 4090 * 1卡

4. 实验设计与实现

4.1 期望探究的问题

本实验主要希望探究以下问题：

1. SVD与SGNS两种词向量方法在语义相似度任务上的性能差异
2. PPMI变换对SVD方法性能的影响
3. 使用SVD向量初始化对SGNS训练的影响

4.2 实验设计

本人设计了两组对照实验：

1. **实验一**：无PPMI变换的SVD vs. 有PPMI变换的SVD
2. **实验二**：使用随机初始化的SGNS vs. 使用SVD向量初始化的SGNS

为了方便修改代码，我实际进行的两个等效实验是：

1. **实验一**：无PPMI变换的SVD + 使用随机初始化的SGNS
2. **实验二**：有PPMI变换的SVD + 使用SVD向量初始化的SGNS

实验使用WordSim-353数据集评估，通过计算词向量的余弦相似度与人工标注相似度的Spearman等级相关系数和Pearson相关系数来衡量性能。

4.3 实验实现

4.3.1 数据预处理与词表构建

首先进行语料预处理，包括小写化、分词，并构建词表：

代码块

```
1  def preprocess_corpus(file_path):
2      with open(file_path, 'r', encoding='utf-8') as f:
3          text = f.read().lower()
4
5      # 按空格和标点进行分词
6      tokens = re.findall(r'\b\w+\b', text)
7      return tokens
8
9  def build_vocab(tokens, min_count=MIN_COUNT):
10     word_counts = Counter(tokens)
11     filtered_words = {word: count for word, count in word_counts.items()
12                       if count >= min_count}
13
14     vocab = {'<UNK>': 0} # 添加未知词标记
15     id2word = {0: '<UNK>'}
16
17     for i, (word, _) in enumerate(sorted(filtered_words.items(), key=lambda x:
18                                         x[1], reverse=True)):
19         vocab[word] = i + 1
20         id2word[i + 1] = word
21
22     # 计算词频率
23     total_words = sum(filtered_words.values())
24     word_freq = {word: count / total_words for word, count in
25                  filtered_words.items()}
26     return vocab, id2word, word_freq
```

词表构建时，我们过滤掉频次低于阈值(MIN_COUNT=5)的词，并加入未知词标记 `<UNK>`。同时，我们计算每个词的归一化频率，用于后续SGNS的负采样和下采样。

4.3.2 SVD方法实现

SVD方法主要包括三个步骤：构建共现矩阵、应用PPMI变换和SVD降维。

1. 构建共现矩阵

共现矩阵采用稀疏表示，考虑了词之间的距离权重：

代码块

```
1 def build_cooccurrence_matrix(tokens, vocab, window_size=SVD_WINDOW_SIZE,
2   save_path=None):
3     # ... [省略部分代码]
4     for i, target_id in enumerate(word_ids):
5         # 在窗口内计算共现次数
6         for j in range(max(0, i - window_size), min(len(word_ids), i +
7   window_size + 1)):
8             if i == j: # 跳过词本身
9                 continue
10            context_id = word_ids[j]
11
12            # 按照距离加权, 越远的上下文权重越低
13            weight = 1.0 / abs(i - j) # 添加距离权重
14            cooccurrence[(target_id, context_id)] += weight
```

2. PPMI变换

PPMI变换计算了词对的点互信息，并将负值截断为0：

代码块

```
1 def apply_ppmi(cooc_matrix):
2     # 获取词频 (边缘概率)
3     row_sum = np.array(cooc_matrix.sum(axis=1)).flatten()
4     col_sum = np.array(cooc_matrix.sum(axis=0)).flatten()
5
6     # 计算总和用于归一化
7     total_sum = float(cooc_matrix.sum())
8
9     # 避免除以零, 添加一个小的epsilon
10    epsilon = 1e-10
11
12    # PMI计算
13    rows, cols = cooc_matrix.nonzero()
14    num_nonzero = len(rows)
15
16    # 初始化新稀疏矩阵的数据数组
17    new_data = np.zeros(num_nonzero)
18
19    # 计算非零元素的PPMI
20    for idx in range(num_nonzero):
21        i, j = rows[idx], cols[idx]
22        count = cooc_matrix[i, j]
```

```

23
24         # 计算PMI
25         p_i_j = count / total_sum
26         p_i = row_sum[i] / total_sum
27         p_j = col_sum[j] / total_sum
28
29         # 避免log(0)通过添加epsilon
30         pmi = np.log((p_i_j + epsilon) / ((p_i * p_j) + epsilon))
31
32         # 应用PPMI (将负PMI值设为0)
33         new_data[idx] = max(0, pmi)

```

3. SVD降维

SVD降维使用TruncatedSVD并对词向量进行归一化：

代码块

```

1  def apply_svd(cooc_matrix, n_components=EMBEDDING_DIM):
2      print("正在对共现矩阵应用PPMI变换...")
3      cooc_matrix = apply_ppmi(cooc_matrix)
4      print("正在进行奇异值分解...")
5      svd = TruncatedSVD(n_components=n_components, random_state=42)
6      word_vectors = svd.fit_transform(cooc_matrix)
7      # 对词向量进行归一化处理
8      print("正在对词向量进行归一化...")
9      norm = np.linalg.norm(word_vectors, axis=1, keepdims=True)
10     # 避免除以0
11     norm[norm == 0] = 1.0
12     word_vectors = word_vectors / norm

```

4.3.3 SGNS方法实现

SGNS方法基于PyTorch实现，主要包括模型定义和训练过程。

1. 模型定义

模型包括两个嵌入层，分别表示目标词和上下文词：

代码块

```

1  class Word2Vec(nn.Module):
2      def
3      __init__
4      (self, vocab_size, embedding_size=EMBEDDING_DIM, padding_idx=0,
5       pretrained_embeddings=None):
6          super(Word2Vec, self).

```

```

6  __init__
7  ()
8      self.vocab_size = vocab_size
9      self.embedding_size = embedding_size
10
11     # 创建目标词和上下文词的嵌入层
12     self.target_embeddings = nn.Embedding(vocab_size, embedding_size,
padding_idx=padding_idx)
13     self.context_embeddings = nn.Embedding(vocab_size, embedding_size,
padding_idx=padding_idx)
14
15     # 如果提供预训练的词向量, 使用它们初始化嵌入层
16     if pretrained_embeddings is not None:
17         pretrained_embeddings_tensor =
torch.FloatTensor(pretrained_embeddings)
18
19         self.target_embeddings.weight.data.copy_(pretrained_embeddings_tensor)
20
21         self.context_embeddings.weight.data.copy_(pretrained_embeddings_tensor)
22     else:
23         # 随机初始化
24         bound = 0.5 / embedding_size
25         nn.init.uniform_(self.target_embeddings.weight.data[1:], -bound,
bound)
26         nn.init.uniform_(self.context_embeddings.weight.data[1:], -bound,
bound)

```

SGNS模型实现了负采样损失函数:

代码块

```

1  class SGNS(nn.Module):
2      def
3      __init__(
4      (self, embedding, vocab_size, vocab=None, word_freq=None, n_negs=20):
5          # ... [省略部分代码]
6
7      def forward(self, iword, owords):
8          # ... [省略部分代码]
9
10         # 负采样
11         if self.weights is not None:
12             weights = self.weights.to(device)
13             nwords = torch.multinomial(weights, batch_size * context_size *
self.n_negs, replacement=True).view(batch_size, -1)
14         else:

```

```

15         nwords = torch.randint(1, self.vocab_size, (batch_size,
context_size * self.n_negs), device=device)
16
17         # 获取嵌入
18         iword_embeds = self.embedding.forward_target(iword).unsqueeze(2)
19         oword_embeds = self.embedding.forward_context(owords)
20         nword_embeds = self.embedding.forward_context(nwords).neg()
21
22         # 计算正样本损失和负样本损失
23         pos_score = torch.bmm(oword_embeds, iword_embeds).squeeze()
24         pos_loss = torch.log(torch.sigmoid(pos_score))
25
26         neg_score = torch.bmm(nword_embeds, iword_embeds).squeeze()
27         neg_score = neg_score.view(batch_size, context_size, self.n_negs)
28         neg_loss = torch.mean(torch.sum(torch.log(torch.sigmoid(neg_score)),
dim=2), dim=1)
29
30         # 总损失
31         return -torch.mean(pos_loss + neg_loss)

```

2. 训练过程

训练过程包括数据加载、优化和模型保存：

代码块

```

1  def train_sgns_model(tokens, vocab, id2word, word_freq,
pretrained_embeddings=None, ...):
2      # ... [省略部分代码]
3
4      for epoch in range(epochs):
5          # 每个epoch重新创建数据集，应用新的下采样
6          dataset = PermutedSubsampledCorpus(tokens, vocab, word_freq,
window_size, subsample_t)
7          dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True,
num_workers=4)
8
9          total_loss = 0
10         progress_bar = tqdm(dataloader, desc=f"Epoch {epoch+1}/{epochs}")
11
12         for batch_idx, (target, context) in enumerate(progress_bar):
13             target = target.to(device)
14             context = context.unsqueeze(1).to(device)
15
16             loss = model(target, context)
17
18             optimizer.zero_grad()

```



```

19         loss.backward()
20         optimizer.step()
21         # ... [省略日志记录代码]

```

数据集实现了下采样策略：

代码块

```

1  class PermutedSubsampledCorpus(Dataset):
2      def
3  __init__
4  (self, tokens, vocab, word_freq, window_size=SGNS_WINDOW_SIZE,
    ss_t=SAMPLE_THRESHOLD):
5      # ... [省略部分代码]
6
7      # 计算下采样概率
8      self.subsampling_rates = {}
9      for word, freq in word_freq.items():
10         # 计算保留概率: 1 - sqrt(t/f)
11         self.subsampling_rates[word] = 1.0 - np.sqrt(ss_t / freq)
12
13     def _create_training_data(self):
14         data = []
15         for i, target_word in enumerate(self.tokens):
16             # ... [省略部分代码]
17
18             # 下采样频繁词
19             if target_word in self.subsampling_rates and random.random() >
                self.subsampling_rates[target_word]:
20                 continue

```

4.3.4 评估方法

使用WordSim-353数据集评估词向量性能，计算余弦相似度与人工标注相似度的Spearman和Pearson相关系数：

代码块

```

1  def evaluate_wordsim(eval_file, word_to_vec_svd, word_to_vec_sgns):
2      人工评分列表 = []
3      svd相似度列表 = []
4      sgns相似度列表 = []
5
6      # ... [省略文件读取代码]
7
8      for line in lines:

```

```

9      # ... [省略词对提取代码]
10
11     # 计算SVD相似度
12     if word1 in word_to_vec_svd and word2 in word_to_vec_svd:
13         sim_svd = cosine_similarity(word_to_vec_svd[word1],
word_to_vec_svd[word2])
14     else:
15         sim_svd = 0.0
16
17     # 计算SGNS相似度
18     if word1 in word_to_vec_sgns and word2 in word_to_vec_sgns:
19         sim_sgns = cosine_similarity(word_to_vec_sgns[word1],
word_to_vec_sgns[word2])
20     else:
21         sim_sgns = 0.0
22     # ... [省略相关系数计算代码]

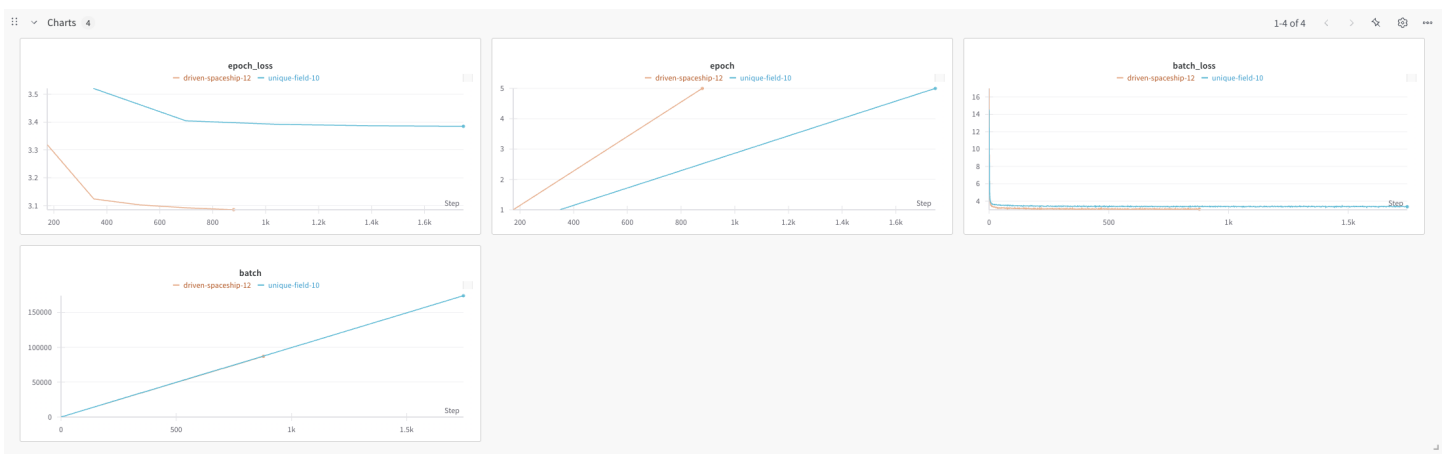
```

5. 实验结果与结论

5.1 实验训练情况

实验训练使用wandb进行可视化，可访问下址查看训练信息：

下图是两次实验中SGNS模型训练过程中损失函数的变化。可以观察到，模型训练初期损失下降迅速，随后趋于平稳持续下降，训练情况良好。



5.2 实验结果分析

通过对比两组实验结果，我们有以下重要发现：

实验一结果（无PPMI的SVD vs. 随机初始化的SGNS）：

方法	Spearman 系数	p值	Pearson系 数	p值

SVD	0.298177	0	0.294968	0
SGNS	0.347647	0	0.397292	0

实验二结果（有PPMI的SVD vs. SVD初始化的SGNS）：

方法	Spearman 系数	p值	Pearson系 数	p值
SVD	0.627386	0	0.617709	0
SGNS	0.659183	0	0.621081	0

应实验要求，这里给出SVD方法的关键参数：

- 非零奇异值总数: 1000（由于矩阵秩过大，只计算了前1000个）
- 选取的奇异值个数: 200
- 选取的奇异值之和: 2632325.203294085
- 全部奇异值之和: 3038919.695242039
- 选取奇异值占总奇异值的比例: 0.8662

5.3 实验发现

无PPMI的SVD相关性较低（Spearman=0.298），而有PPMI的SVD相关性显著提高（Spearman=0.627），增幅超过0.33，这表明PPMI变换对提取有意义的共现关系至关重要，通过抑制高频词的影响而突出语义相关的共现模式

随机初始化的SGNS性能一般（Spearman=0.348），而使用SVD向量初始化的SGNS性能大幅提升（Spearman=0.659），增幅超过0.31， 这表明高质量的预训练词向量可以显著提升SGNS模型的性能，为模型提供更好的优化起点

在无PPMI情况下，SGNS（0.348）优于SVD（0.298），在使用PPMI和SVD初始化的情况下，SGNS（0.659）略优于纯SVD方法（0.627），这表明结合全局统计（SVD+PPMI）和局部上下文学习（SGNS）的优势，可以获得更好的词向量表示。实验二中SGNS的出色表现证明了"先全局后局部"的训练策略效果显著，同时，SVD捕获可以全局统计关系，二SGNS进一步精调局部上下文关系，两者形成互补，能够获得更好的词向量表示结果。

5.4 结论

本实验通过对比SVD和SGNS两种词向量方法及其组合，得出以下关键结论：

1. PPMI变换对共现矩阵的处理至关重要，能显著提升SVD方法的性能

2. SVD初始化能大幅提升SGNS模型的学习效果，表明高质量的预训练词向量可以为神经网络模型提供更好的优化起点
3. 结合SVD和SGNS的两阶段训练策略能够获得最佳性能，综合了全局统计和局部上下文学习的优势
4. 不同方法在不同类型的语义关系上表现各异，但总体而言，结合使用能够获得更为全面和准确的词向量表示